TUTORIAL 02

Spring Boot Recap

How the container works, the core stereotypes, and how to build a REST application from an empty project to a running API.

IoC / DI | @Component | Stereotypes | REST | application.yml | Validation

Table of Contents

1. What Spring Boot Is

Spring Boot is a framework that lets you build production-grade Java applications with almost no boilerplate. It sits on top of the **Spring Framework** and adds three things: **auto-configuration**, an **embedded server**, and **opinionated starter dependencies**.

Inversion of Control & Dependency Injection

The core idea of Spring is **Inversion of Control (IoC)**. Instead of your objects creating their own collaborators with `new`, a container creates them and hands them over. Giving an object its dependencies from the outside is called **Dependency Injection (DI)**.

```
// WITHOUT Spring: the class builds its own dependencies (tightly coupled)
public class OrganizerService {
    private final OrganizerRepository repo = new OrganizerRepository(); // hard-wired
}

// WITH Spring: dependencies are handed in -- easy to swap and to test
public class OrganizerService {
    private final OrganizerRepository repo;
    public OrganizerService(OrganizerRepository repo) {   // injected
        this.repo = repo;
    }
}
```

> **TIP**
>
> DI is why you can inject a real `StripeGateway` in production and a `FakePaymentGateway` in tests without changing a line of the service — the exact pattern from Tutorial 01 (abstraction + polymorphism), now automated by the container.

2. Creating an Application

Start from Spring Initializr

The fastest way to begin is `start.spring.io`. Pick Maven, Java 21, the latest Spring Boot 3.x, and add starters (Web, JPA, Validation...). It generates a ready-to-run project.

The POM — starters do the heavy lifting

A **starter** is a curated bundle of dependencies. Adding one line pulls in everything you need for a feature, all version-aligned by the parent POM.

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
</parent>

<dependencies>
    <!-- REST + embedded Tomcat + JSON in one line -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <!-- JPA + Hibernate (Tutorial 03) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
</dependencies>
```

The main class

One annotated class boots the whole application. This is the real entry point of the event-ticketing project:

```
@SpringBootApplication          // = @Configuration + @EnableAutoConfiguration + @Component
public class EventTicketingApplication {
    public static void main(String[] args) {
        SpringApplication.run(EventTicketingApplication.class, args);
    }
}
```

Part of `@SpringBootApplication`	What it does
`@ComponentScan`	Scans this package (and sub-packages) for your beans.
`@EnableAutoConfiguration`	Configures Tomcat, Jackson, JPA, etc. based on what's on the classpath.
`@Configuration`	Lets the class itself declare `@Bean` methods.

> **NOTE**
> Because component scanning starts at the main class's package, keep `EventTicketingApplication` at the **root package** (`com.eventticketing`) so every sub-package is discovered.

Run it with `./mvnw spring-boot:run` — an embedded Tomcat starts on port 8080. No WAR file, no external server to install.

3. Beans & the Container

A **bean** is simply an object that Spring creates and manages. The **ApplicationContext** is the container that holds every bean and knows how to wire them together.

The lifecycle at startup

1. Component scan finds every class annotated as a bean.
2. Spring works out the dependency graph (who needs whom).
3. It instantiates each bean, injecting its dependencies in the right order.
4. Beans are singletons by default — **one shared instance** per type.

Scope	Meaning
`singleton` *(default)*	One instance for the whole application.
`prototype`	A new instance every time it is requested.
`request` / `session`	One per HTTP request / session (web apps).

> **WATCH OUT**
>
> Singleton beans are shared across all threads, so **keep them stateless**. Store per-request data in method parameters or local variables, never in a field of a service.

4. Core Stereotypes

Stereotype annotations mark a class as a bean *and* describe its role. They are all specialisations of `@Component` — Spring treats them the same for wiring, but they document intent (and enable role-specific behaviour, like exception translation on repositories).

Annotation	Layer / role
`@Component`	Generic Spring-managed bean.
`@Service`	Business-logic layer (use cases, orchestration).
`@Repository`	Data-access layer; adds DB exception translation.
`@Controller` / `@RestController`	Web layer; handles HTTP requests.
`@Configuration`	Declares `@Bean` factory methods.

A @Service from the project

```
@Service
public class OrganizerService {

    private final OrganizerRepository organizerRepository;

    public OrganizerService(OrganizerRepository organizerRepository) {
        this.organizerRepository = organizerRepository;
    }

    @Transactional(readOnly = true)
    public OrganizerResponse get(Long id) {
        return OrganizerResponse.from(
            organizerRepository.findById(id)
                .orElseThrow(() -> ResourceNotFoundException.of("Organizer", id)));
    }
}
```

@Configuration + @Bean for third-party objects

You can't annotate a class you don't own. Declare it as a bean in a `@Configuration` class instead — like the project's `ClockConfig`:

```java
@Configuration
public class ClockConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();   // now injectable anywhere
    }
}
```

TIP

Injecting a `Clock` bean instead of calling `Instant.now()` directly makes time controllable in tests — another win from programming to an abstraction.

5. Dependency Injection Done Right

Prefer constructor injection

```
@Service
public class EventService {

    private final EventRepository events;
    private final HallService halls;

    // No @Autowired needed: a single constructor is auto-wired.
    public EventService(EventRepository events, HallService halls) {
        this.events = events;
        this.halls  = halls;
    }
}
```

Style	Verdict
Constructor	✓ Preferred. Fields can be `final`; dependencies are explicit and testable.
Setter	Only for genuinely optional dependencies.
Field (`@Autowired` on a field)	✗ Avoid. Hides dependencies and can't be `final`; hard to unit-test.

> **NOTE**
> When two beans implement the same interface, tell Spring which to use with `@Primary` (a default) or `@Qualifier("beanName")` at the injection point.

6. Configuration

Externalise anything that changes between environments — ports, URLs, timeouts — into `application.yml` (or `.properties`).

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/eventticketing
    username: app
    password: secret

reservation:
  hold-duration: 10m        # custom app property
  max-seats-per-booking: 8
```

Reading config: @Value vs @ConfigurationProperties

```
// Single value
@Value("${{server.port}}")
private int port;
```

```
// A whole typed group -- the project uses this style (@ConfigurationPropertiesScan
// on the main class enables it).
@ConfigurationProperties(prefix = "reservation")
public record ReservationProperties(Duration holdDuration, int maxSeatsPerBooking) { }
```

Profiles

Profiles let one build behave differently per environment. Put overrides in `application-dev.yml` / `application-prod.yml` and activate one:

```
java -jar app.jar --spring.profiles.active=prod
```

```
@Bean
@Profile("dev")            // only created when the "dev" profile is active
public DataSeeder seeder() { ... }
```

> **TIP**
>
> Never commit real secrets to `application.yml`. Reference environment variables instead: `password: ${{DB_PASSWORD}}`.

7. Building a REST API

Now the payoff: turn a service into an HTTP API. A REST controller maps HTTP verbs and URLs to Java methods and (de)serialises JSON automatically.

The controller

```
@RestController                        // @Controller + @ResponseBody (returns JSON)
@RequestMapping("/api/organizers")     // base path for every method
public class OrganizerController {

    private final OrganizerService organizerService;

    public OrganizerController(OrganizerService organizerService) {
        this.organizerService = organizerService;
    }

    @PostMapping                        // POST /api/organizers
    public ResponseEntity<OrganizerResponse> create(
            @Valid @RequestBody CreateOrganizerRequest request) {
        return ResponseEntity
                .status(HttpStatus.CREATED)          // 201
                .body(organizerService.create(request));
    }

    @GetMapping("/{{id}}")                 // GET /api/organizers/42
    public OrganizerResponse get(@PathVariable Long id) {
        return organizerService.get(id);   // serialised to JSON, HTTP 200
    }
}
```

Annotation	Binds to
`@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping`	HTTP method + path
`@PathVariable`	A `{id}` segment in the URL
`@RequestParam`	A query-string parameter (`?status=ON_SALE`)
`@RequestBody`	The JSON request body → a Java object
`ResponseEntity<T>`	Full control of status code + headers + body

DTOs — never expose entities directly

Use small **records** for requests and responses. This decouples your API from the database schema and controls exactly what is exposed:

```
public record CreateOrganizerRequest(
        @NotBlank String name,
        @Email String email,
        String phone) { }

public record OrganizerResponse(Long id, String name, String email, String phone) {
    // map an entity to its response DTO
    public static OrganizerResponse from(Organizer o) {
        return new OrganizerResponse(o.getId(), o.getName(), o.getEmail(), o.getPhone());
    }
}
```

Validation

Annotate DTO fields with constraints and add `@Valid` in the controller. Spring rejects a bad body with **400 Bad Request** before your code runs.

```
@NotBlank   // not null and not empty
@Email      // valid email shape
@Min(1) @Max(8)
@NotNull
```

Centralised error handling

Rather than try/catch in every controller, handle exceptions in one place with `@RestControllerAdvice` — exactly how the project's `GlobalExceptionHandler` works:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> notFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)   // 404
                .body(new ApiError("NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> invalid(MethodArgumentNotValidException ex) {
        return ResponseEntity.badRequest()                   // 400
                .body(new ApiError("VALIDATION", "request is invalid"));
    }
}
```

This is where the exceptions from Tutorial 01 pay off: services throw a meaningful `RuntimeException`, and one advice class maps each type to the right HTTP status. Clean separation, no clutter.

The request flow, end to end

```
HTTP request
   -> DispatcherServlet          (Spring front controller)
   -> Controller  (@RestController)   parse path/body, validate
   -> Service     (@Service)          business rules, @Transactional
   -> Repository  (@Repository)       talk to the database  (Tutorial 03)
   <- Response DTO -> JSON  -> HTTP response
```

8. Cheat Sheet

BOOTSTRAPPING

- `@SpringBootApplication` on the root class; `SpringApplication.run(...)` in `main`.
- Add features via **starters** in the POM; run with `./mvnw spring-boot:run`.

STEREOTYPES

`@Service`	Business logic
`@Repository`	Data access
`@RestController`	HTTP/JSON endpoints
`@Configuration` + `@Bean`	Wire objects you don't own

DEPENDENCY INJECTION

- Use **constructor injection** with `final` fields. No `@Autowired` needed for a single constructor.
- Disambiguate with `@Qualifier` / `@Primary`.

REST

- `@GetMapping` / `@PostMapping` + `@PathVariable` / `@RequestParam` / `@RequestBody`.
- Validate with `@Valid` + constraint annotations on record DTOs.
- Handle errors once in a `@RestControllerAdvice`.

> **TIP**
>
> **Next:** Tutorial 03 fills in the repository layer — how JPA & Hibernate turn your entities into database rows and back.